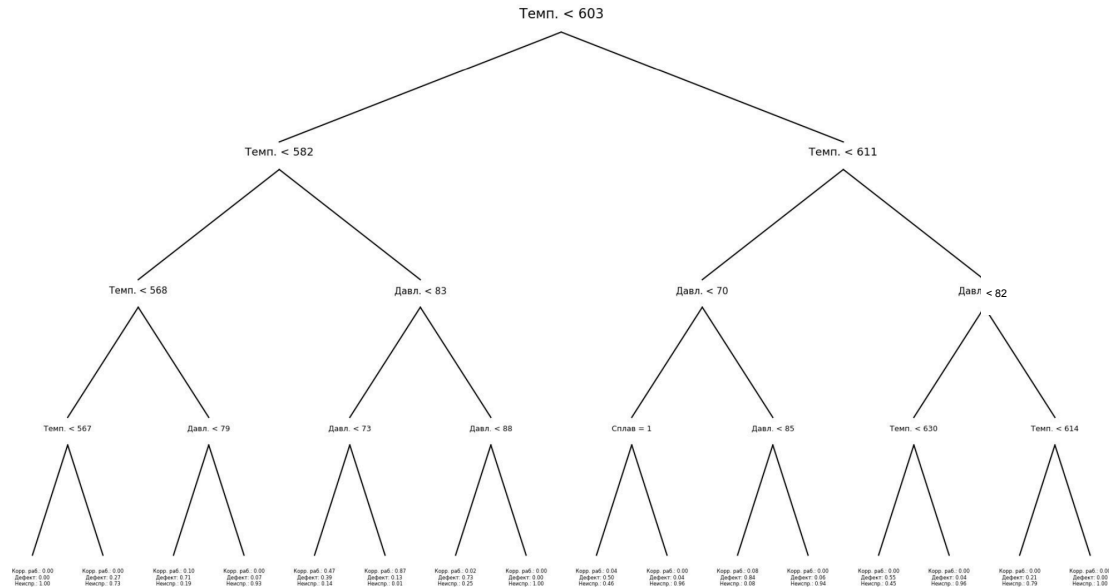


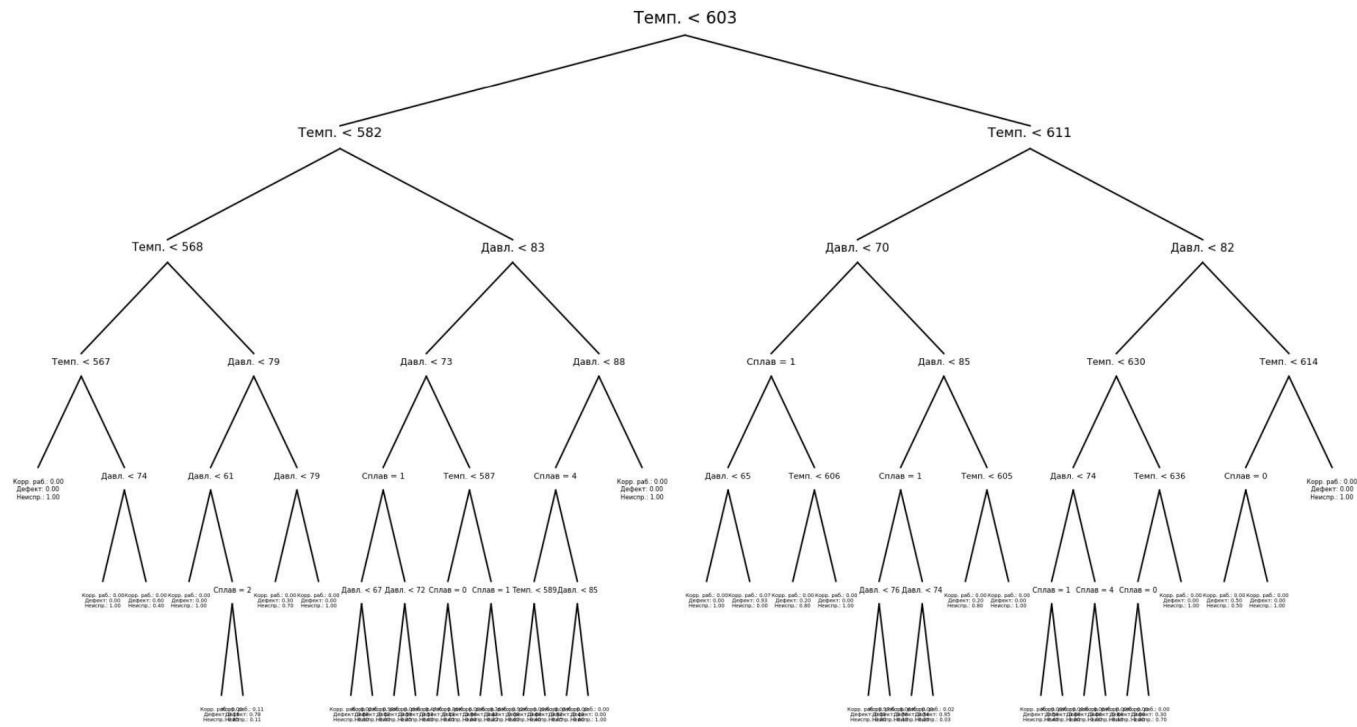
Decision Trees

Rule-based methods

- Set of simple rules (*predicates*)
- Can be represented as a *tree*
- Non-leaf nodes - rules, leaves - final predictions
- Usually have form “*feature j* $\leq v$ ”

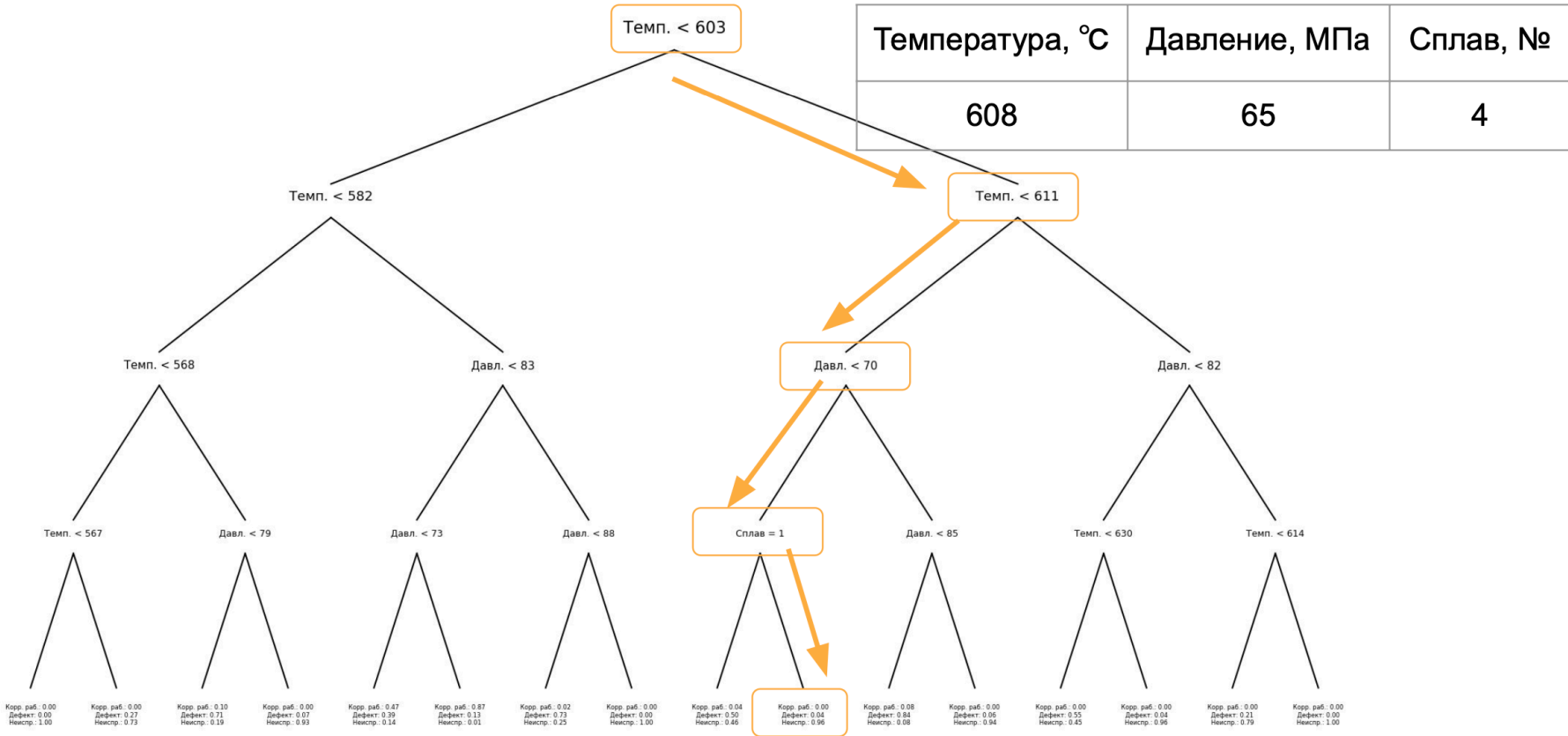


Rule-based methods



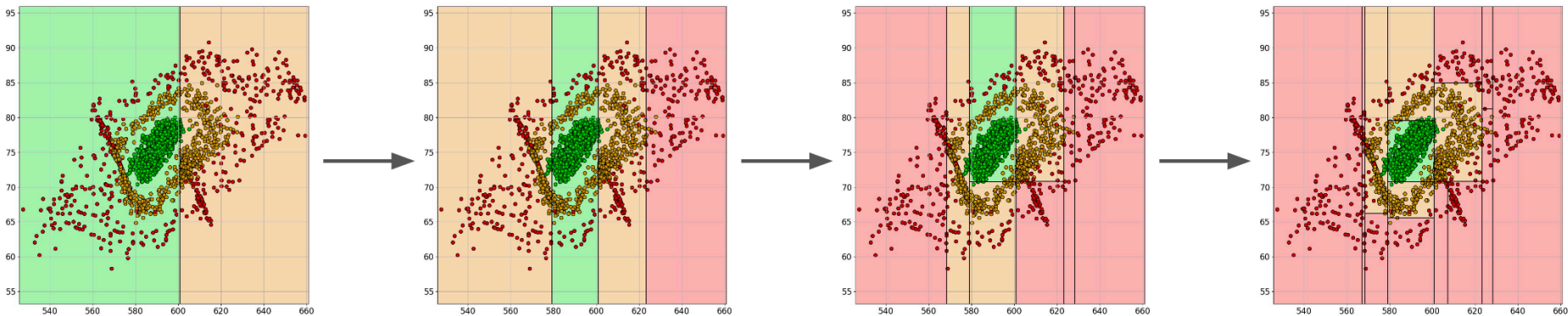
Hard to build manually, want to find rules *automatically*

How to predict?



How to build?

Idea: choose predicate which optimally splits the feature space, then partition our dataset using it and create subtrees recursively



How to choose the best predicate?

Solution: determine which predicate allows to extract *the most information* about the value of the objective function

How to build?

- Let $I(x)$ be a function which estimates *impurity* (uncertainty) of objective function in the current partition
- Then for any predicate we can calculate the *information gain* which we can obtain using it:

$$IG(x_l, x_r) = \frac{|x_l \cup x_r|}{|X|} I(x_l \cup x_r) - \frac{|x_l|}{|X|} I(x_l) - \frac{|x_r|}{|X|} I(x_r)$$

- We aim to **maximize** the information gain

How to build?

Need to estimate *impurity* I .

Options:

- *Misclassification Error*:

$$I_E(X) = 1 - \max_y \left\{ \frac{|x_i : y_i = y|}{|X|} \right\}$$

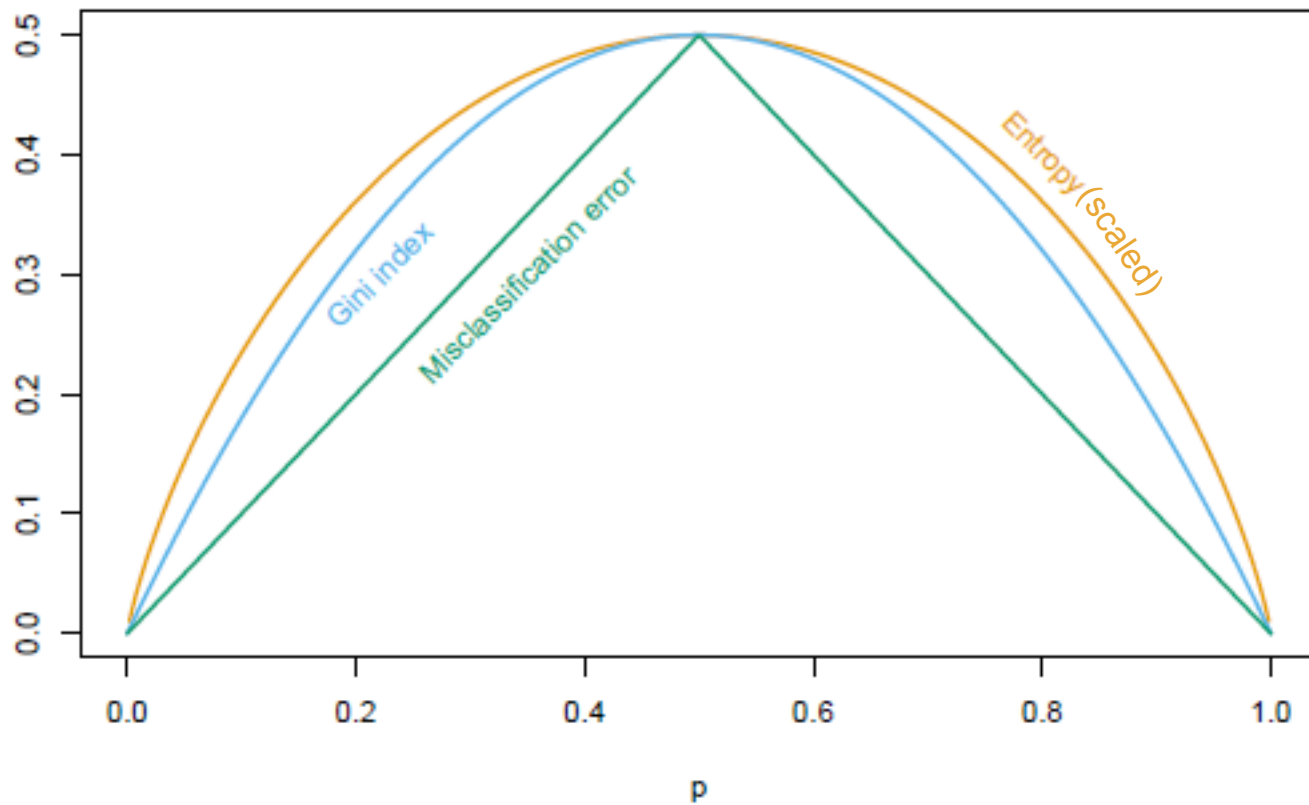
- *Entropy*:

$$I_H(X) = \mathbb{E}(-\log p(y)) = - \sum_{y \in Y} p(y) \log p(y) = - \sum_{y \in Y} \frac{|x_i : y_i = y|}{|X|} \times \log \left(\frac{|x_i : y_i = y|}{|X|} \right)$$

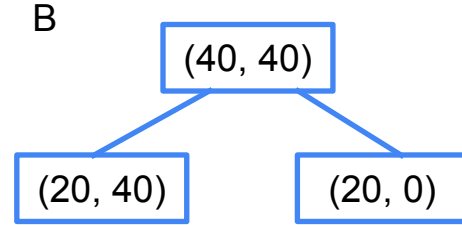
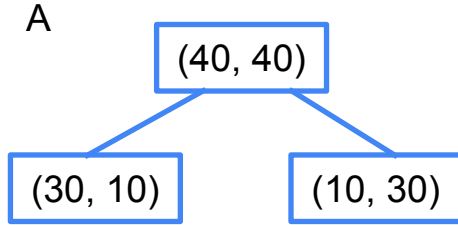
- *Gini Index*:

$$I_G(X) = \mathbb{E}_y(1 - p(y)) = \sum_{y \in Y} p(y)(1 - p(y)) = \sum_{y \in Y} \frac{|x_i : y_i = y|}{|X|} \cdot \frac{|x_y : y_y \neq y|}{|X|}$$

How to build?



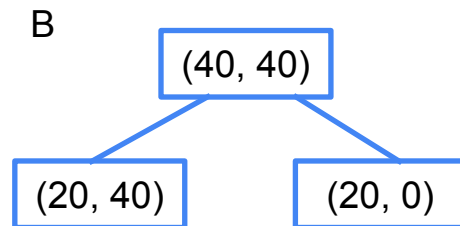
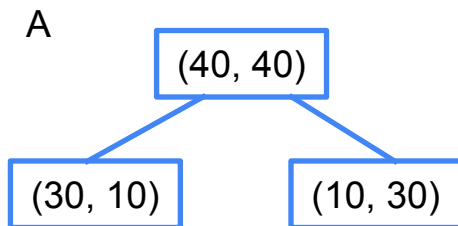
Example



$$IG_E(A) = 1 \cdot 0.5 - 0.5 \cdot 0.25 - 0.5 \cdot 0.25 = 0.25$$

$$IG_E(B) = 1 \cdot 0.5 - 0.75 \cdot 0.33 - 0.25 \cdot 0 = 0.25$$

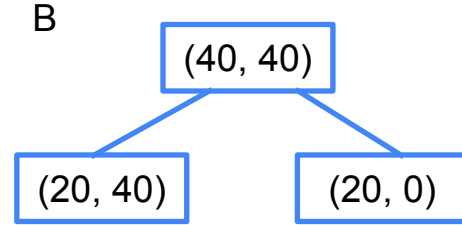
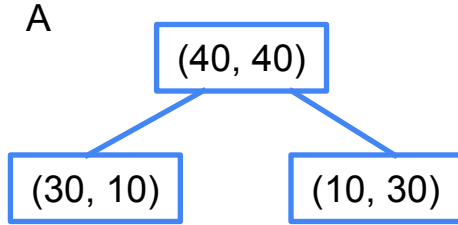
Example



$$IG_H(A) = 1 \cdot 1 - 0.5 \cdot 0.81 - 0.5 \cdot 0.81 = 0.19$$

$$IG_H(B) = 1 \cdot 1 - 0.75 \cdot 0.92 - 0.25 \cdot 0 = 0.31$$

Example



$$IG_G(A) = 1 \cdot 0.5 - 0.5 \cdot 0.375 - 0.5 \cdot 0.375 = 0.125$$

$$IG_E(B) = 1 \cdot 0.5 - 0.75 \cdot 0.44 - 0.25 \cdot 0 = 0.167$$

ID3, C4.5, CART

CART - Classification And Regression Trees

1. If all objects in the node belong to the same class or the number of observations in a node is low -> create a leaf with class probabilities
2. For all features i
 - 2.1. For all values v_i
 - 2.1.1. Calculate Information Gain for predicate $x[i] < v_i$, compare with best value.
3. For the best partition
 - 3.1. Separate X into X_l and X_r according to the best predicate
 - 3.2. Recursively build left and right subtree

Classification **A**nd **R**egression Trees (CART)

- Gini Index for classification

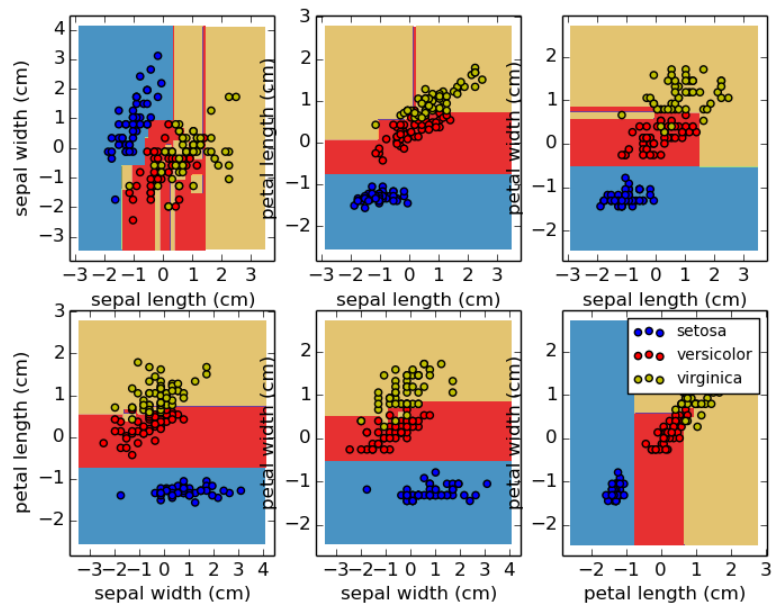
$$I_G(X) = \sum_{y \in Y} \frac{|x_i : y_y = y|}{|X|} \cdot \frac{|x_y : y_y \neq y|}{|X|}$$

- Variance for regression

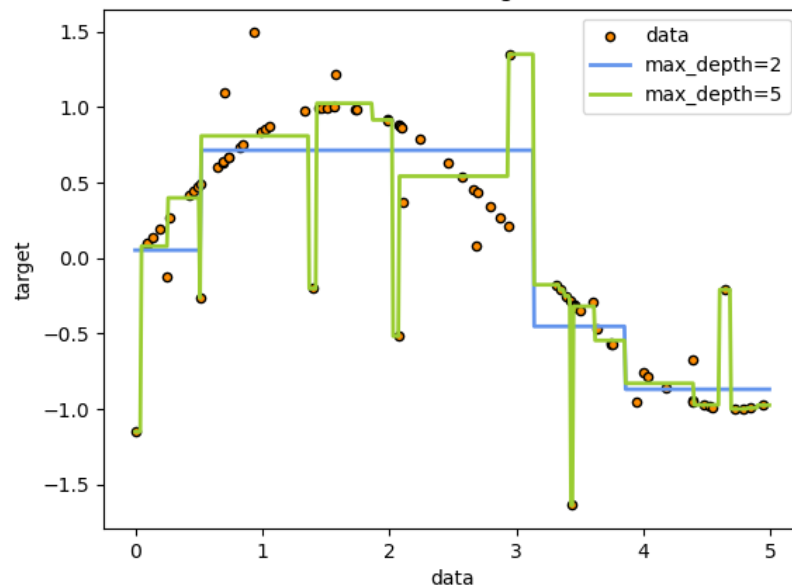
$$I_v(X) = \mathbb{E}_{x_i} (y_i - \bar{y})^2 = \sum_{x_i \in X} (y_i - \bar{y})^2$$

Example

Decision surface of a decision tree using paired features



Decision Tree Regression



Advantages of decision trees

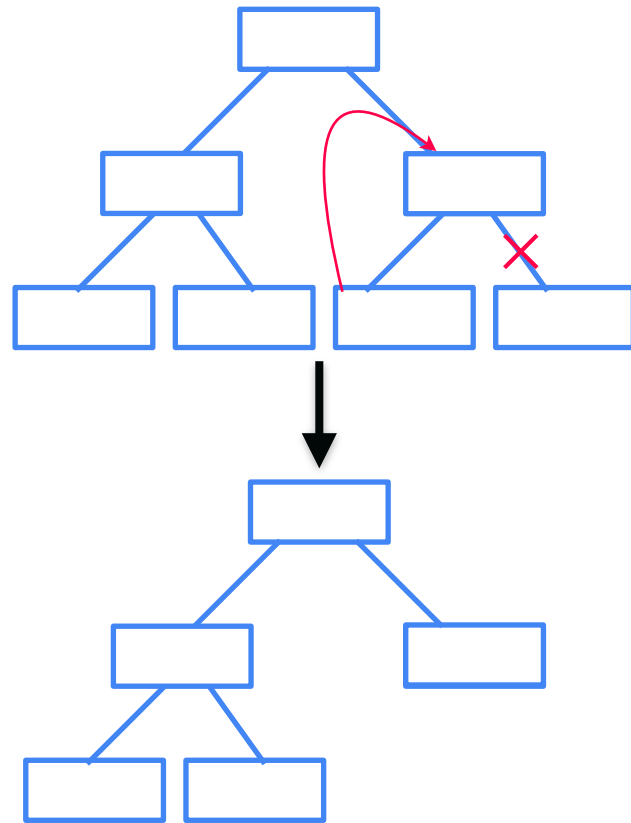
- Can work with both numerical and *categorical* data
- Interpretability
 - Can look at the whole tree (only if the tree is not deep)
 - Can look at the *reason* a particular decision was made (path from the root to the leaf)
 - Can compute *feature importance* (usually when trees are in an *ensemble*) - how many times a feature was used to make a partition
- *Resilience* - robust to outliers
- Almost no need for preprocessing
 - No need to scale
 - Can handle missing values during train
 - Ignore while calculating information gain, assign points to both left and right subtrees
 - Build *surrogate predicates* - use different feature which imply similar partition

Regularization in decision trees

- Limit depth of the tree
- Limit min number of observations in leaf/node
- Limit number of leaves
- **Pruning**: remove useless branches of the tree

How to know which branches are useless?

- Use *validation set* to determine and remove useless branches



Pruning

- Remove branches which are not visited by any point of the validation set
- *Minimum error pruning*: remove branches while validation error is not increasing
- Cost complexity pruning: remove branches while validation error is not increasing by more than α .

In other words, introduce a new, cumulative error function:

$$R_{\alpha}(T) = R(T) + \alpha \cdot |T|$$

$R(T)$ - sum of misclassification errors at each leaf

Speeding up decision trees

Time complexity of naïve CART implementation:

$$\mathcal{O}(N_{features} \cdot N_{samples}^3)$$

- At each node we loop over $N_{features}$ features;
- Go over all possible thresholds (max $N_{samples}$);
- For each threshold split the data in $\mathcal{O}(N_{samples})$, compute the information gain in $\mathcal{O}(N_{samples})$
- Total number of nodes: $\mathcal{O}(N_{samples})$

Speeding up decision trees

- Sort the feature values at each node, keep track of number of labels in each split -> can find best split in $\mathcal{O}(N_{samples} \log N_{samples})$.

$$\mathcal{O}(N_{features} N_{samples}^2 \log N_{samples})$$

- Presorting and keeping the order of examples over all features

$$\mathcal{O}(N_{features} N_{samples}^2)$$

- Binning samples by features

$$\mathcal{O}(N_{features} N_{samples} N_{values})$$

- For each node choose random subset of features

$$\mathcal{O}(N_{subset} N_{samples} N_{values})$$

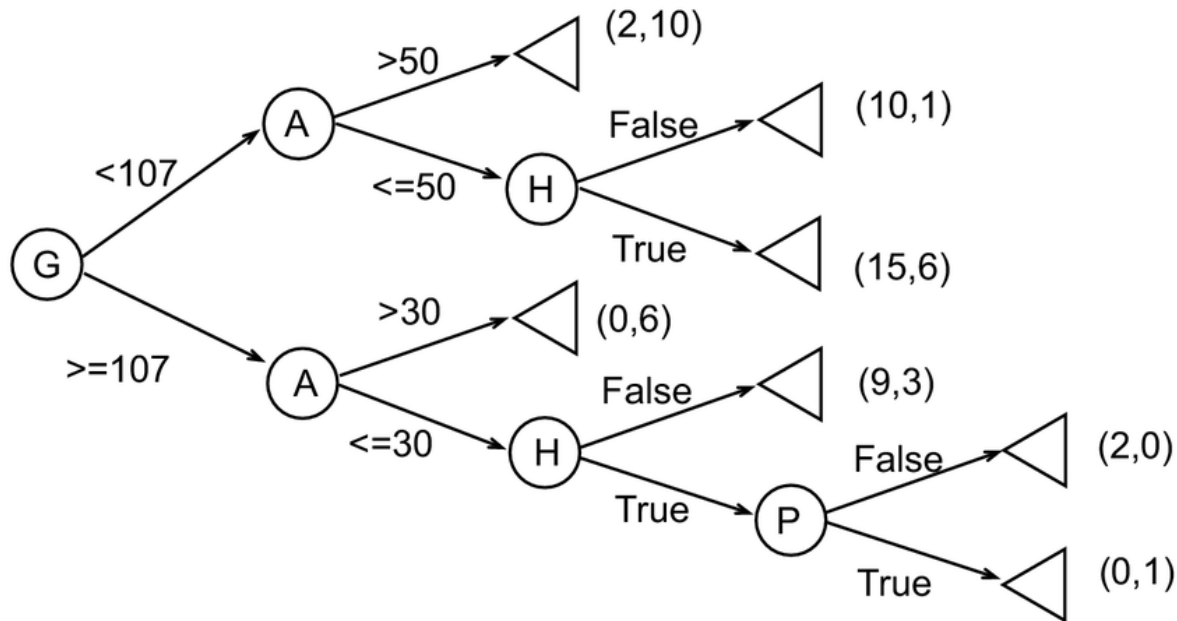
- Parallelize everything

$$\mathcal{O}\left(\frac{N_{subset} N_{samples} N_{values}}{N_{threads}}\right)$$

Oblivious Decision Trees

Idea: use the same feature for all nodes at the same level

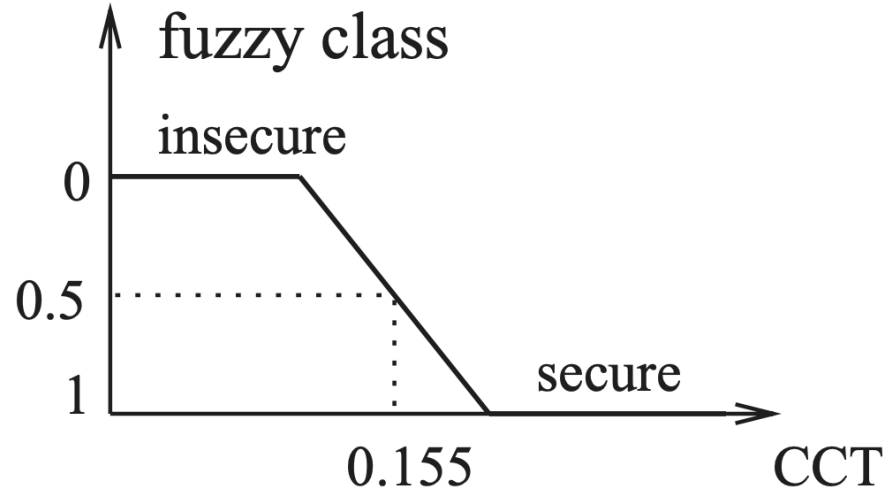
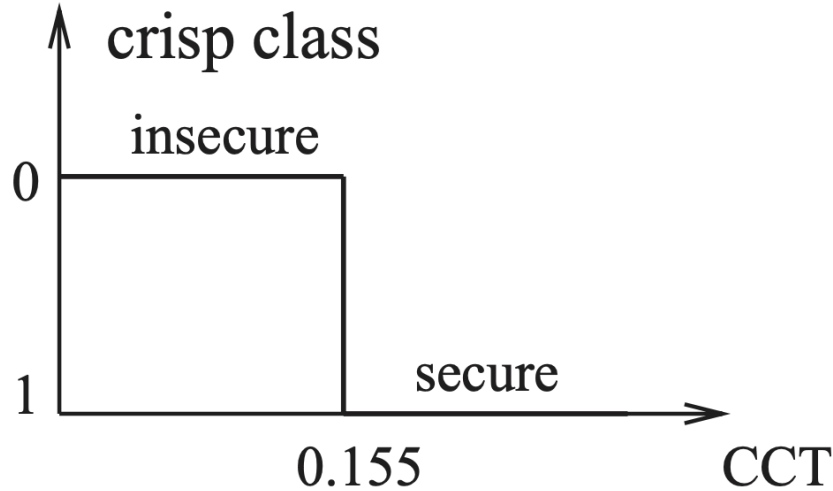
- Faster training
- Feature selection
- Easy to parallelize at runtime
- Balanced
- Regularization



Fuzzy Decision Trees

If a point is close to threshold, “split” it between branches with some weights.

At the end aggregate the probabilities in all leaves it appeared in with the according weights.



0.110